

REPORTE DE PRÁCTICA DE LABORATORIO

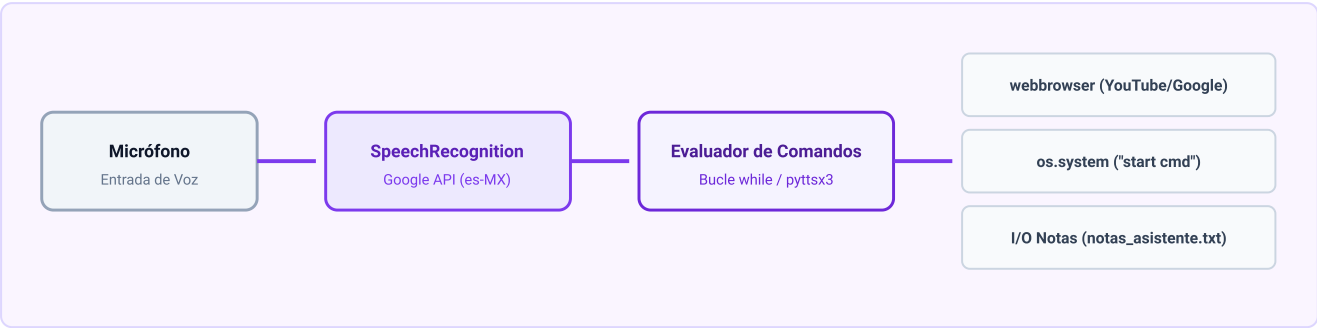
Práctica 10: Asistente Virtual Inteligente en Python (SpeechRecognition & pyttsx3)

ALUMNO	MATERIA	FECHA
Luis Angel Diaz Dominguez	Interfaces de Usuario	27 de Septiembre, 2026

1. OBJETIVOS

- Construir un **Asistente Virtual de escritorio en Python** capaz de procesar instrucciones de voz y ejecutar tareas automatizadas del sistema operativo y navegación web.
- Utilizar las librerías `speech_recognition` (STT vía Google Speech API) y `pyttsx3` (Síntesis de voz fuera de línea).
- Implementar automatizaciones como: apertura de sitios web (YouTube, Google), consulta de la hora actual, ejecución de aplicaciones del sistema (CMD) y persistencia de notas escritas.

2. DIAGRAMA DE ARQUITECTURA DEL ASISTENTE VIRTUAL



3. CÓDIGO FUENTE DEL PROYECTO (`ASISTENTE.PY`)

asistente.py - Módulos e Inicialización

```

import speech_recognition as sr
import pyttsx3
import datetime
import webbrowser
import os

# Configuración de Voz (Texto a Voz)
engine = pyttsx3.init()
voices = engine.getProperty('voices')

# Selección de voz en español (el índice varía según el sistema)
for v in voices:
    if "spanish" in v.name.lower():
        engine.setProperty('voice', v.id)
        break

engine.setProperty('rate', 180)

def speak(text):
    print(f"Asistente: {text}")
    engine.say(text)
    engine.runAndWait()

def listen():
    recognizer = sr.Recognizer()
    with sr.Microphone() as source:

```

```

def greet_me():
    hour = int(datetime.datetime.now().hour)
    if 0 <= hour < 12:
        speak("Buenos días.")
    elif 12 <= hour < 18:
        speak("Buenas tardes.")
    else:
        speak("Buenas noches.")
    speak("Soy tu asistente. ¿En qué puedo ayudarte hoy?")

def run_assistant():
    greet_me()
    while True:
        query = listen()

        if 'abrir youtube' in query:
            webbrowser.open("https://www.youtube.com")
            speak("Abriendo YouTube")

        elif 'hora' in query:
            strTime = datetime.datetime.now().strftime("%H:%M:%S")
            speak(f"La hora actual es {strTime}")

        elif 'busca en google' in query:
            speak("¿Qué quieres buscar?")
            search = listen()
            if search != "none":
                webbrowser.open(f"https://www.google.com/search?q={search}")
                speak(f"Esto es lo que encontré para {search}")

        elif 'terminal' in query:
            speak("Abriendo la terminal de Windows")
            os.system("start cmd")

        elif 'escribe una nota' in query:
            speak("¿Qué debería anotar?")
            note_content = listen()
            if note_content != "none":
                with open("notas_asistente.txt", "a", encoding="utf-8") as f:
                    fecha = datetime.datetime.now()
                    f.write(f"{fecha}: {note_content}\n")
                speak("Nota guardada localmente.")

        elif 'adiós' in query or 'dormir' in query:
            speak("Entendido. Estaré aquí si me necesitas. Hasta pronto.")
            break

if __name__ == "__main__":
    run_assistant()

```

4. CONCLUSIONES Y DIAGNÓSTICO DE EJECUCIÓN

- **Integración Multimodular:** La combinación de pyttsx3 para síntesis de audio local y speech_recognition con la API de Google permite procesar comandos en lenguaje natural en tiempo real con alta exactitud en español (es-MX).
- **Análisis del Error en Captura (PyAudio):** Durante la ejecución de prueba documentada en la captura, se registró la excepción `AttributeError / Could not find PyAudio`. Esto se resuelve instalando el controlador de audio subyacente ejecutando `pip install pyaudio` en el entorno virtual de Python.
- **Automatización del Sistema:** El asistente resuelve tareas de automatización comunes como el acceso directo a la terminal del sistema operativo y la persistencia de archivos de texto con registro de fecha y hora